



**UNIVERSIDADE DE FORTALEZA**  
**VICE REITORIA DE GRADUAÇÃO**  
**CENTRO DE CIÊNCIAS TECNOLÓGICAS**  
**TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Bruno Lourenço dos Santos  
Gabriel Cavalcanti Esmeraldo  
Juliana Inacio Teixeira  
Marcelo Henrique Marques Ribeiro  
Matheus Teixeira Gomes de Carvalho

**DOCUMENTAÇÃO TÉCNICA**

Sistema Prisma

FORTALEZA

2026

Bruno Lourenço dos Santos  
Gabriel Cavalcanti Esmeraldo  
Juliana Inacio Teixeira  
Marcelo Henrique Marques Ribeiro  
Matheus Teixeira Gomes de Carvalho

## **DOCUMENTAÇÃO TÉCNICA**

Sistema Prisma

Este documento contém a documentação técnica do Sistema Prisma desenvolvido na componente curricular N392 - Projeto Aplicado Plataformas Web como requisito para obtenção de nota.

Supervisor: Prof. Bruno Lopes, Me

FORTALEZA

2026

## SUMÁRIO

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1. INTRODUÇÃO.....</b>                        | <b>4</b>  |
| 1.1. CONTEXTO E JUSTIFICATIVA.....               | 4         |
| 1.2. OBJETIVOS.....                              | 4         |
| 1.3. ESCOPO E DELIMITAÇÃO.....                   | 5         |
| <b>2. ENGENHARIA DE REQUISITOS.....</b>          | <b>6</b>  |
| 2.1. REQUISITOS FUNCIONAIS (RFs).....            | 6         |
| 2.2. REQUISITOS NÃO FUNCIONAIS (RNFs).....       | 7         |
| <b>3. PROJETO E ARQUITETURA DO SOFTWARE.....</b> | <b>8</b>  |
| 3.1. ARQUITETURA GERAL.....                      | 8         |
| 3.2. PROJETO DO BANCO DE DADOS.....              | 11        |
| 3.3. PROJETO DE API.....                         | 13        |
| <b>4. TECNOLOGIAS E FERRAMENTAS.....</b>         | <b>14</b> |
| 4.1. STACK DE TECNOLOGIAS.....                   | 14        |
| 4.2. FERRAMENTAS DE DESENVOLVIMENTO.....         | 15        |
| <b>5. IMPLEMENTAÇÃO E RESULTADOS.....</b>        | <b>16</b> |
| 5.1. TELAS DO SISTEMA.....                       | 16        |
| <b>6. AMBIENTE E GUIA DE IMPLANTAÇÃO.....</b>    | <b>22</b> |
| 6.1. REQUISITOS DO AMBIENTE.....                 | 22        |
| 6.2. PROCESSO DE IMPLANTAÇÃO.....                | 22        |
| 6.3. ACESSO À APLICAÇÃO IMPLANTADA.....          | 24        |
| <b>7. CONCLUSÃO.....</b>                         | <b>25</b> |
| 7.1. TRABALHOS FUTUROS.....                      | 25        |
| 7.2. LIÇÕES APRENDIDAS.....                      | 25        |

# 1. INTRODUÇÃO

## 1.1. CONTEXTO E JUSTIFICATIVA

O ecossistema de jogos eletrônicos é atualmente fragmentado entre grandes plataformas, como Steam, Xbox, PlayStation e RetroAchievements, cada uma possuindo seus próprios sistemas fechados de troféus, rankings e estatísticas. Para o jogador moderno, que frequentemente transita entre consoles e o PC, essa dispersão cria um cenário de dados difusos: não existe um local único onde ele possa visualizar seu desempenho global ou consolidar sua progressão de forma unificada. Essa fragmentação impede que o usuário tenha uma percepção clara de sua "carreira" no mundo dos games, resultando em conquistas que ficam esquecidas em perfis isolados.

O Sistema Prisma surge como a solução para centralizar essa experiência, permitindo que jogadores unifiquem suas conquistas de múltiplas plataformas em um único perfil dinâmico. Além de oferecer uma visão holística do desempenho do usuário, a plataforma permite a personalização do perfil para destacar conquistas favoritas e promove a interação social por meio de um sistema de "seguidores", conectando amigos e comunidades. O Prisma é essencial tanto para o jogador casual, que busca organizar suas memórias de jogo, quanto para o jogador profissional (e-athletes e streamers), funcionando como um portfólio online que proporciona visibilidade e comprova suas habilidades e histórico técnico em um mercado cada vez mais competitivo.

## 1.2. OBJETIVOS

O objetivo geral deste projeto é desenvolver uma plataforma unificada que integre e centralize o histórico de conquistas, troféus e estatísticas de diferentes ecossistemas de jogos (Steam, Xbox, PlayStation e RetroAchievements), consolidando a identidade digital do jogador em um único perfil social e profissional.

Dito isso, os objetivos específicos do projeto são:

- Implementar integração via API com as principais plataformas de jogos para a importação automática de dados de conquistas e progresso em tempo real.
- Desenvolver um perfil de usuário customizável que permite ao jogador selecionar e exibir suas conquistas de maior raridade ou importância (exibição de "badges" ou troféus favoritos).
- Criar um sistema de seguidores, permitindo que usuários sigam outros perfis de jogadores
- Estabelecer um sistema de ranking que pontua o desempenho do jogador somando os dados de todas as suas contas conectadas.

### 1.3. ESCOPO E DELIMITAÇÃO

Nesta seção, definimos as fronteiras do Sistema Prisma, detalhando as funcionalidades que serão entregues e estabelecendo claramente o que não faz parte dos objetivos deste desenvolvimento.

#### **Escopo:**

- **Integração e exposição de perfis:** Conexão com plataformas externas (Steam, Xbox, PlayStation, RetroAchievements) para visualização centralizada de estatísticas.
- **Gestão de conquistas:** Sincronização e exibição do histórico de troféus obtidos pelo usuário em diferentes ecossistemas.
- **Personalização de perfil:** Funcionalidade para o usuário destacar suas conquistas favoritas.
- **Módulo de interação social:** Funcionalidade para seguir outros jogadores, ser seguido e navegar entre perfis da comunidade.
- **Navegação e busca:** Sistema de busca de usuários na rede Prisma.

#### **Delimitação (Fora do Escopo):**

- **Execução de jogos:** O sistema não é um emulador ou plataforma de execução; não será possível jogar através do Prisma.

- **E-commerce e transações:** Não haverá comercialização de jogos, itens in-game ou qualquer sistema de compra e venda (B2C/C2C).
- **Transmissão de áudio e vídeo:** O sistema não funcionará como plataforma de streaming ou canal de transmissão ao vivo (como Twitch ou YouTube).

## 2. ENGENHARIA DE REQUISITOS

### 2.1. REQUISITOS FUNCIONAIS (RFs)

Esta seção descreve os Requisitos Funcionais do Sistema Prisma.

| ID   | Nome do Requisito              | Descrição                                                                                                                         |
|------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| RF01 | Criar conta                    | O sistema deve permitir que o usuário crie uma conta informando dados como nome completo, email, nickname e senha                 |
| RF02 | Verificar email                | O sistema deve validar o email do usuário durante o cadastro por meio de um código de verificação enviado ao endereço informado   |
| RF03 | Realizar login                 | O sistema deve permitir que o usuário já cadastrado realize login ao informar email e senha                                       |
| RF04 | Vincular contas externas       | O sistema deve permitir que o usuário conecte suas contas da Steam, Xbox, PlayStation e RetroAchievements via integração de API   |
| RF05 | Sincronizar jogos e conquistas | O sistema deve importar e atualizar a lista de jogos e troféus obtidos nas plataformas conectadas                                 |
| RF06 | Personalizar troféus fixados   | O sistema deve permitir que o usuário selecione até 4 troféus específicos para destacar no topo de seu perfil                     |
| RF07 | Visualizar perfil              | O sistema deve permitir que o usuário tenha uma visão geral do seu perfil, incluindo desempenho geral conquistas e jogos recentes |
| RF08 | Seguir usuários                | O sistema deve permitir que um jogador siga outros usuários e seja seguido                                                        |

|      |                            |                                                                                                    |
|------|----------------------------|----------------------------------------------------------------------------------------------------|
| RF09 | Visualizar ranking geral   | O sistema deve gerar uma classificação de jogadores                                                |
| RF10 | Personalizar perfil social | O sistema deve permitir a edição de foto de perfil, bio e nickname/username para exibição pública. |

## 2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)

Esta seção descreve os Requisitos Não Funcionais do Sistema Prisma.

### Usabilidade:

- **RNF01:** A interface deve ser responsiva, permitindo que o jogador visualize seu portfólio de forma clara tanto em diferentes tamanhos de tela em desktop.
- **RNF02:** Antes de desvincular qualquer conta de plataforma (Steam, Xbox, PSN, RetroAchievements), o sistema exibe um modal de confirmação com a mensagem "Tem certeza que deseja desvincular sua conta?" e um aviso de que a ação não pode ser desfeita. O modal possui botões de confirmar e cancelar, além de suporte a fechamento por clique fora e tecla Escape, prevenindo perda acidental do histórico de sincronização.
- **RNF03:** Durante a sincronização em lote de múltiplas plataformas, o sistema exibe em tempo real o andamento. Isso é possível através de broadcast via Phoenix PubSub, onde o SyncService emite eventos de progresso que são consumidos pela LiveView sem necessidade de recarregar a página.

### Segurança:

- **RNF04:** O sistema deve implementar um controle de acesso restrito, impedindo a visualização de qualquer tela interna ou funcionalidade sem autenticação prévia, com exceção exclusiva das páginas de login, cadastro de novos usuários e recuperação de senha.
- **RNF05:** Toda requisição a rotas internas (dashboard, perfil, configurações, sincronização) passa pelo plug `:require_authenticated_user`, que verifica a existência de um `current_scope` com usuário válido na sessão.
- **RNF06:** A senha do usuário é submetida ao algoritmo no momento da validação do `changeset`, antes de ser persistida no banco de dados. O campo `password` é definido como virtual (`virtual: true`, `redact: true`) e removido do `changeset` após o hash, garantindo que nunca seja logado ou retornado em queries. O campo `hashed_password` armazena apenas o hash, tornando a senha original irrecuperável mesmo em caso de vazamento do banco.

## Portabilidade:

- **RNF07:** A aplicação utiliza Phoenix LiveView com fallback automático para long-poll em ambientes sem suporte nativo a WebSocket, além de Tailwind CSS v4 para estilização — que gera CSS compatível com os motores de renderização dos principais navegadores. Não há dependência de APIs JavaScript proprietárias ou experimentais, garantindo funcionamento nas duas versões mais recentes de Chrome, Firefox, Safari e Edge.
- **RNF08:** Todo o ambiente de desenvolvimento e produção é containerizado via compose.yml e Dockerfile, utilizando Alpine Linux como imagem base para a aplicação Elixir e PostgreSQL 17 como banco de dados.
- **RNF09:** Todas as configurações sensíveis e específicas de ambiente (credenciais de banco, chave secreta, host, chaves de API de plataformas de jogos) são definidas exclusivamente via variáveis de ambiente no arquivo config/runtime.exs, com validação de presença em produção que impede a inicialização caso alguma variável obrigatória esteja ausente. Isso permite que a mesma imagem Docker seja promovida entre ambientes sem recompilação.

## 3. PROJETO E ARQUITETURA DO SOFTWARE

### 3.1. ARQUITETURA GERAL

O sistema Prisma foi projetado utilizando uma Arquitetura em Camadas (Monolito Modular) ou seja, todo o código roda em uma única aplicação, mas está organizado em módulos independentes (Contexts, Adapters, LiveViews) que podem ser evoluídos separadamente sem bagunçar o restante do sistema:

1. **Camada de Apresentação (Frontend):** Desenvolvida com Phoenix LiveView e estilizada com Tailwind CSS e Heroicons. Esta camada é responsável por fornecer uma interface de usuário rica, reativa e em tempo real. A escolha do LiveView justifica-se pela sua capacidade de renderizar atualizações dinâmicas no lado do servidor e enviá-las via WebSockets, eliminando a necessidade de uma API REST separada e simplificando a sincronização de estado entre cliente e servidor.
2. **Camada de Aplicação (Backend):** Desenvolvida em Elixir com o framework Phoenix v1.8. Esta camada gerencia a lógica de negócio através de Contextos (Accounts, Catalog, Sync), que agrupam funcionalidades relacionadas de forma isolada. Para se comunicar com as plataformas de jogos (Steam, PlayStation, Xbox, RetroAchievements, IGDB), o sistema usa o padrão Adapter — cada plataforma tem um "tradutor" próprio que converte os dados da API externa para o formato interno do Prisma. Um Rate Limiter controla o volume de requisições para evitar bloqueios por excesso de



chamadas. O Elixir foi escolhido por sua alta concorrência e tolerância a falhas nativa (graças à máquina virtual Erlang/OTP), garantindo o processamento de milhares de eventos simultâneos com baixa latência

3. **Camada de Dados (Banco de Dados):** O sistema utiliza o banco de dados PostgreSQL, selecionado por sua robustez e conformidade com as propriedades ACID. A biblioteca Ecto gerencia a persistência, garantindo a integridade dos dados via schemas e changesets, que permitem validações rigorosas antes de qualquer gravação.

Infraestrutura de Suporte:

O servidor HTTP é o Bandit, que substitui o Cowboy como adapter do Phoenix. O sistema conta com:

- Telemetry — coleta de métricas em tempo real.
- Swoosh com adapter Resend — envio de e-mails transacionais (confirmação de cadastro, recuperação de senha).
- DNSCluster — descoberta de nós em ambiente distribuído.

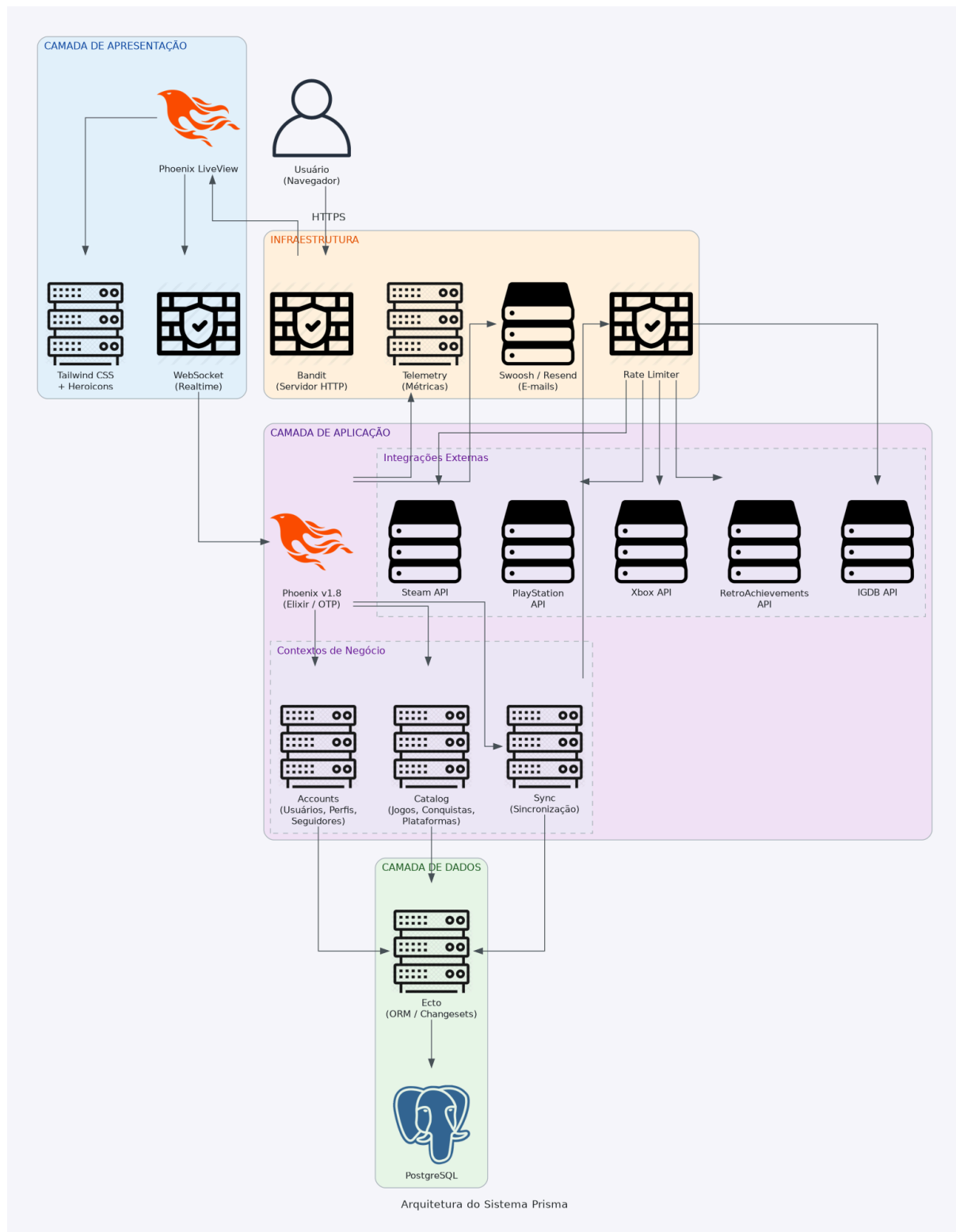
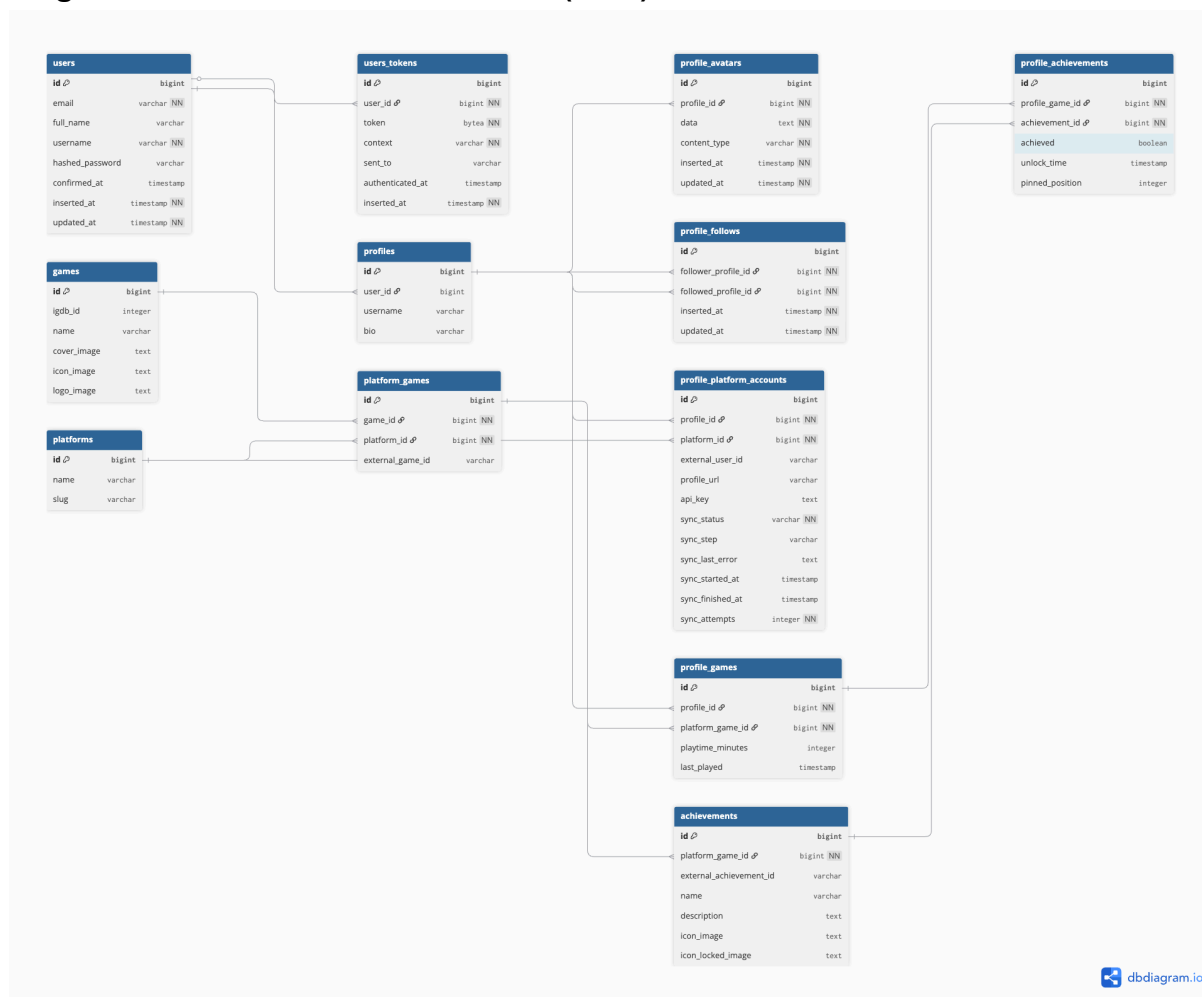


Diagrama da arquitetura

## 3.2. PROJETO DO BANCO DE DADOS

### Diagrama Entidade-Relacionamento (DER):



[\(link do diagrama\)](#)

### Dicionário de Dados (Tabela **profiles**):

| Nome do campo | Tipo de dados | Chave (PK/FK) | Nulo? | Descrição                                     |
|---------------|---------------|---------------|-------|-----------------------------------------------|
| id            | INT IDENTITY  | PK            | Não   | Identificador único gerado automaticamente    |
| userId        | BIG_INT       | FK            | Nao   | Chave estrangeira relacionando a tabela users |

|          |         |   |     |                                              |
|----------|---------|---|-----|----------------------------------------------|
| username | VARCHAR | - | Sim | Nome de usuário para exibição no Prisma      |
| bio      | VARCHAR | - | Sim | Biografia do usuário para exibição no Prisma |

**Dicionário de Dados** (Tabela `profile_platform_accounts`):

| Nome do campo    | Tipo de dados | Chave (PK/FK) | Nulo? | Descrição                                    |
|------------------|---------------|---------------|-------|----------------------------------------------|
| id               | INT IDENTITY  | PK            | Não   | Identificador único da vinculação            |
| profile_id       | INT           | FK            | Sim   | Referência ao perfil do Prisma (profiles.id) |
| platform_id      | INT           | FK            | Sim   | Referência à plataforma (platforms.id)       |
| external_user_id | VARCHAR       | -             | Sim   | ID do usuário na plataforma original         |
| profile_url      | VARCHAR       | -             | Sim   | Link do perfil na plataforma externa         |

**Dicionário de Dados** (Tabela `games`):

| Nome do campo | Tipo de dados | Chave (PK/FK) | Nulo? | Descrição                   |
|---------------|---------------|---------------|-------|-----------------------------|
| id            | INT IDENTITY  | PK            | Não   | Identificador único do jogo |
| igdb_id       | INT           | -             | Sim   | ID de referência            |
| name          | VARCHAR       | -             | Sim   | Título oficial do jogo      |

|             |         |   |     |                        |
|-------------|---------|---|-----|------------------------|
| cover_image | VARCHAR | - | Sim | URL da imagem de capa  |
| icon_image  | VARCHAR | - | Sim | URL da imagem de ícone |
| logo_image  | VARCHAR | - | Sim | URL da imagem de logo  |

### 3.3. PROJETO DE API

O sistema Prisma adota uma arquitetura monolítica com frontend e backend unificados, não possuindo uma API REST separada. Essa decisão arquitetural foi tomada com base nas seguintes premissas:

#### **Por que não foi utilizada uma API REST separada?**

O Prisma é uma aplicação web server-rendered construída com Phoenix LiveView. Nesse modelo, toda a lógica de apresentação e interação roda no servidor, e o navegador recebe apenas HTML atualizado via WebSockets — não via chamadas HTTP tradicionais a uma API. O LiveView elimina a necessidade de:

- Endpoints JSON para consumo do frontend
- Serialização/deserialização de dados entre frontend e backend
- Manutenção de contratos de API (como OpenAPI)
- Duplicação de lógica de validação e estado entre camadas

Dessa forma, não existe uma "API pública" no sentido tradicional. A comunicação entre cliente e servidor é feita de forma transparente pelo próprio framework, através de eventos LiveView (phx-click, phx-submit, phx-change) que disparam funções no servidor e retornam o HTML atualizado.

#### **Pipeline :api reservada para futuras extensões**

Apesar de o sistema não utilizar uma API REST no momento, o arquivo router.ex já define uma pipeline :api configurada para aceitar requisições JSON:

```
pipeline :api do
```

```
  plug :accepts, ["json"]
```

```
end
```

Essa pipeline está intencionalmente reservada para futuras extensões do sistema, como:

- Desenvolvimento de um aplicativo mobile nativo.
- Integração com serviços de terceiros.
- Exposição de endpoints para consumo externo.

## 4. TECNOLOGIAS E FERRAMENTAS

### 4.1. STACK DE TECNOLOGIAS

Nesta seção listamos as stacks de tecnologia do Sistema Prisma, detalhando os principais motivos para as escolhas feitas.

- **Frontend:**
  - **Phoenix LiveView v1.1:** Utilizado para criar uma interface rica e reativa com comunicação em tempo real via WebSockets. Ele permite que o progresso da sincronização de conquistas seja atualizado instantaneamente no navegador sem a complexidade de um framework SPA (Single Page Application).
  - **Tailwind CSS v0.3:** Framework CSS utilitário para estilização rápida e responsiva. Integrado diretamente ao pipeline de assets do Phoenix, garante uma interface moderna com baixo custo de manutenção.
  - **Heroicons v2.2:** Conjunto de ícones otimizados utilizados para compor a identidade visual e sinalizar as diferentes plataformas de jogos no sistema.
- **Backend:**
  - **Elixir v1.15:** Linguagem funcional que utiliza a máquina virtual Erlang (BEAM). Escolhida pela sua alta tolerância a falhas e suporte nativo a processos concorrentes, essencial para realizar múltiplas integrações com APIs externas simultaneamente.
  - **Phoenix Framework v1.8:** Framework web principal utilizado para estruturar a aplicação, gerenciar rotas e organizar a lógica de negócio através de contextos isolados.
  - **Ecto v3.13 (Ecto SQL):** Camada de persistência e consulta de dados. Utilizada para garantir a integridade das informações unificadas e gerenciar as migrações do banco de dados.
  - **Req v0.5:** Biblioteca cliente HTTP moderna utilizada para realizar as requisições às APIs da Steam, PSN e IGDB de forma simplificada e eficiente.
  - **Bandit v1.5:** Servidor HTTP de alto desempenho construído puramente em Elixir, utilizado para hospedar a aplicação com foco em velocidade e conformidade com os padrões modernos da web.

- **Bcrypt Elixir v3.0:** Utilizado para o hashing seguro de senhas, garantindo a proteção das credenciais dos usuários no banco de dados.
- **Banco de Dados:**
  - **PostgreSQL:** Banco de dados relacional robusto utilizado para armazenar o catálogo de jogos e os relacionamentos complexos entre perfis e conquistas, garantindo consistência através de transações ACID.

## 4.2. FERRAMENTAS DE DESENVOLVIMENTO

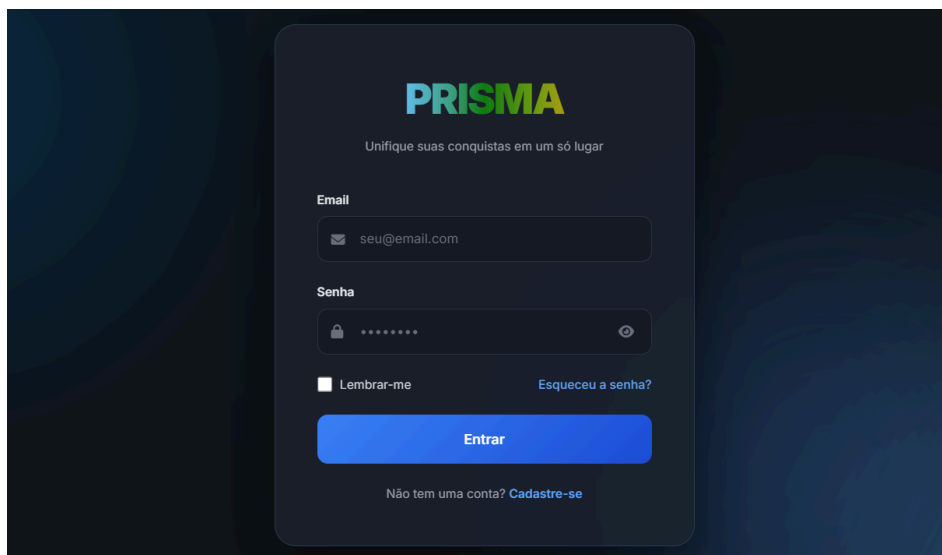
Abaixo listamos as ferramentas que apoiaram o processo de desenvolvimento, do sistema passando pela escrita do código até o gerenciamento das tarefas da equipe.

- **IDE:** Visual Studio Code foi a IDE padrão para toda a equipe, devido à sua leveza e extensibilidade para o ecossistema Elixir.
- **Controle de Versão:** Git, com o repositório hospedado no GitHub. Adotamos um fluxo de trabalho baseado em branches independentes para cada membro desenvolver suas tarefas antes de realizar o merge na *main*.
- **Gerenciamento de Projeto:** Foi utilizado o quadro Kanban na área "Projetos" do GitHub, com as colunas "Backlog", "To do", "In Progress" e "Done" para organizar as sprints e o desenvolvimento das funcionalidades.
- **Ferramenta de API:** O terminal interativo do Elixir (IEx) em conjunto com a biblioteca Req foi utilizado para testar os endpoints das APIs diretamente no ambiente de desenvolvimento, dispensando ferramentas gráficas externas.
- **Containerização:** Docker e Docker Compose foram utilizados para orquestrar os containers da aplicação e do banco de dados PostgreSQL, garantindo a padronização do ambiente entre todos os desenvolvedores.

## 5. IMPLEMENTAÇÃO E RESULTADOS

### 5.1. TELAS DO SISTEMA

Figura 1: Tela de login



A tela de login do sistema PRISMA apresenta o logotipo da marca no topo, seguido por uma mensagem de boas-vindas. Abaixo, há campos para inserir o e-mail e a senha, com ícones de email e cadeado para orientação. O campo de senha possui um ícone para alternar a visibilidade. Abaixo dos campos, há uma opção de "Lembrar-me" com uma caixa de seleção e um link para "Esqueceu a senha?". Um botão azul "Entrar" está centralizado, e no rodapé da caixa, há um link para "Cadastre-se" para quem não possui uma conta.

**PRISMA**

Unifique suas conquistas em um só lugar

**Email**

**Senha**



☐ Lembrar-me [Esqueceu a senha?](#)

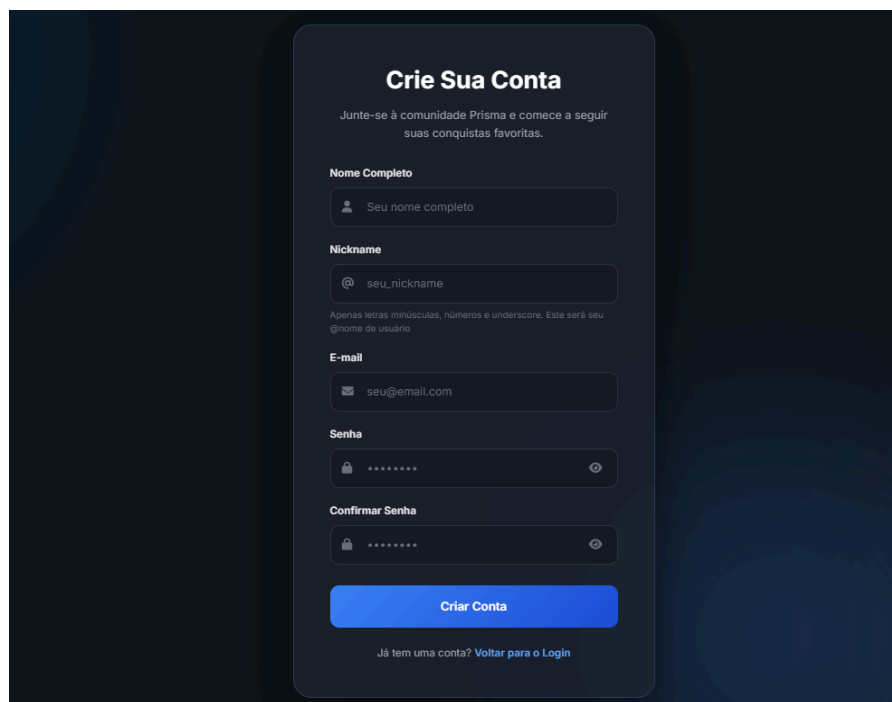
**Entrar**

Não tem uma conta? [Cadastre-se](#)

*A tela de login garante o acesso seguro através de autenticação por e-mail e senha.*

---

Figura 2: Tela de cadastrar usuário



A tela de cadastro, intitulada "Crie Sua Conta", orienta o usuário a se juntar à comunidade PRISMA. Ela solicita o nome completo, um nickname (com uma dica de formatação: apenas letras minúsculas, números e underline) e o e-mail. Os campos de senha e confirmação de senha possuem ícones para alternar a visibilidade. Um botão azul "Criar Conta" está no final, e um link "Voltar para o Login" é fornecido para quem já possui uma conta.

**Crie Sua Conta**

Junte-se à comunidade Prisma e comece a seguir suas conquistas favoritas.

**Nome Completo**

**Nickname**

Apenas letras minúsculas, números e underscore. Este será seu @nome de usuário

**E-mail**

**Senha**



**Confirmar Senha**



**Criar Conta**

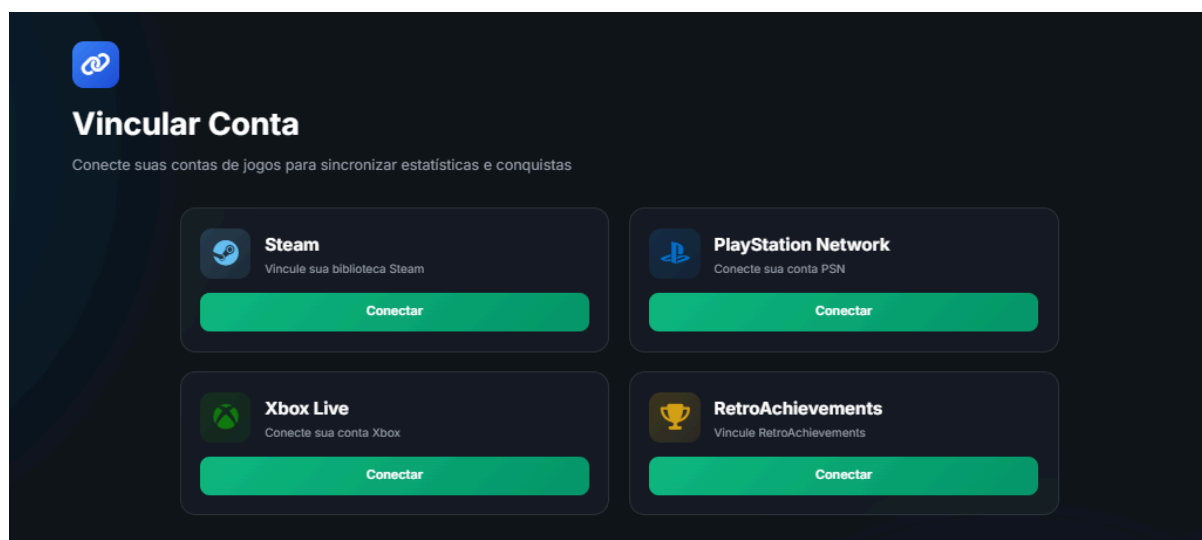
Já tem uma conta? [Voltar para o Login](#)

*A tela de cadastro permite que um novo usuário se cadastre*



---

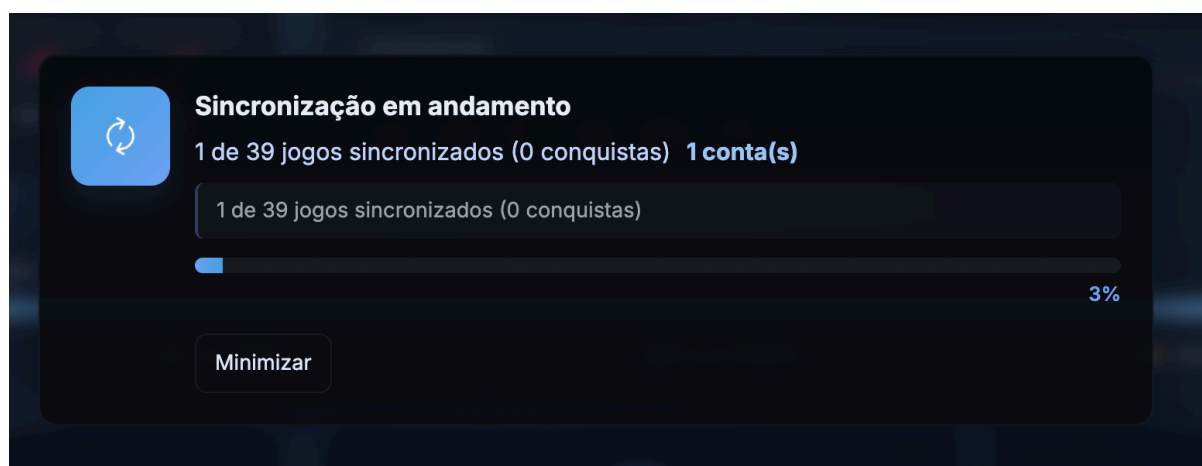
**Figura 3: Tela de vincular contas externas**



*A tela de vinculação de contas aparece após o cadastro do novo usuário solicitando vincular ao menos uma plataforma.*

---

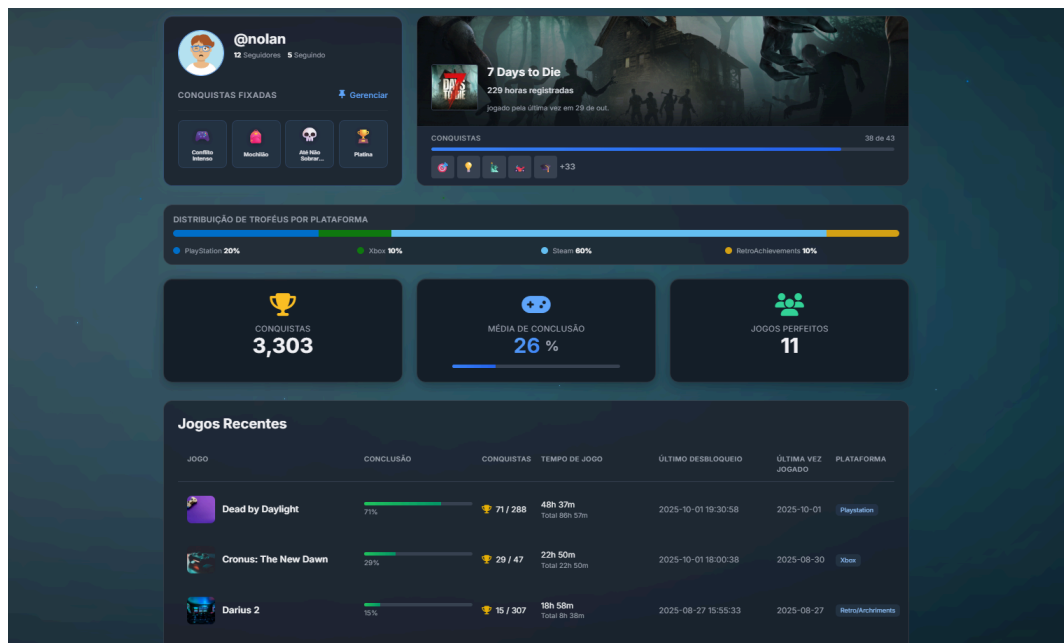
**Figura 4: Tela de sincronizar conquistas**



*A tela de sincronização aparece após a vinculação de uma conta externa, enquanto o usuário aguarda a importação de suas conquistas.*

---

Figura 5: Tela de perfil



*Tela onde o usuário tem uma visão geral do seu perfil*

Figura 6: Tela de personalizar perfil social

**Editar Perfil**

Foto de Perfil

Escolher Foto

JPG, PNG, GIF ou WebP. Máximo 2MB.

**Nome Completo**

marcelo henrique

Seu nome real. Será exibido no seu perfil e em outras áreas do sistema.

**Nickname**

@ marcelogplays

Apenas letras minúsculas, números e underscore. O @ será exibido automaticamente no seu perfil.

**Bio**

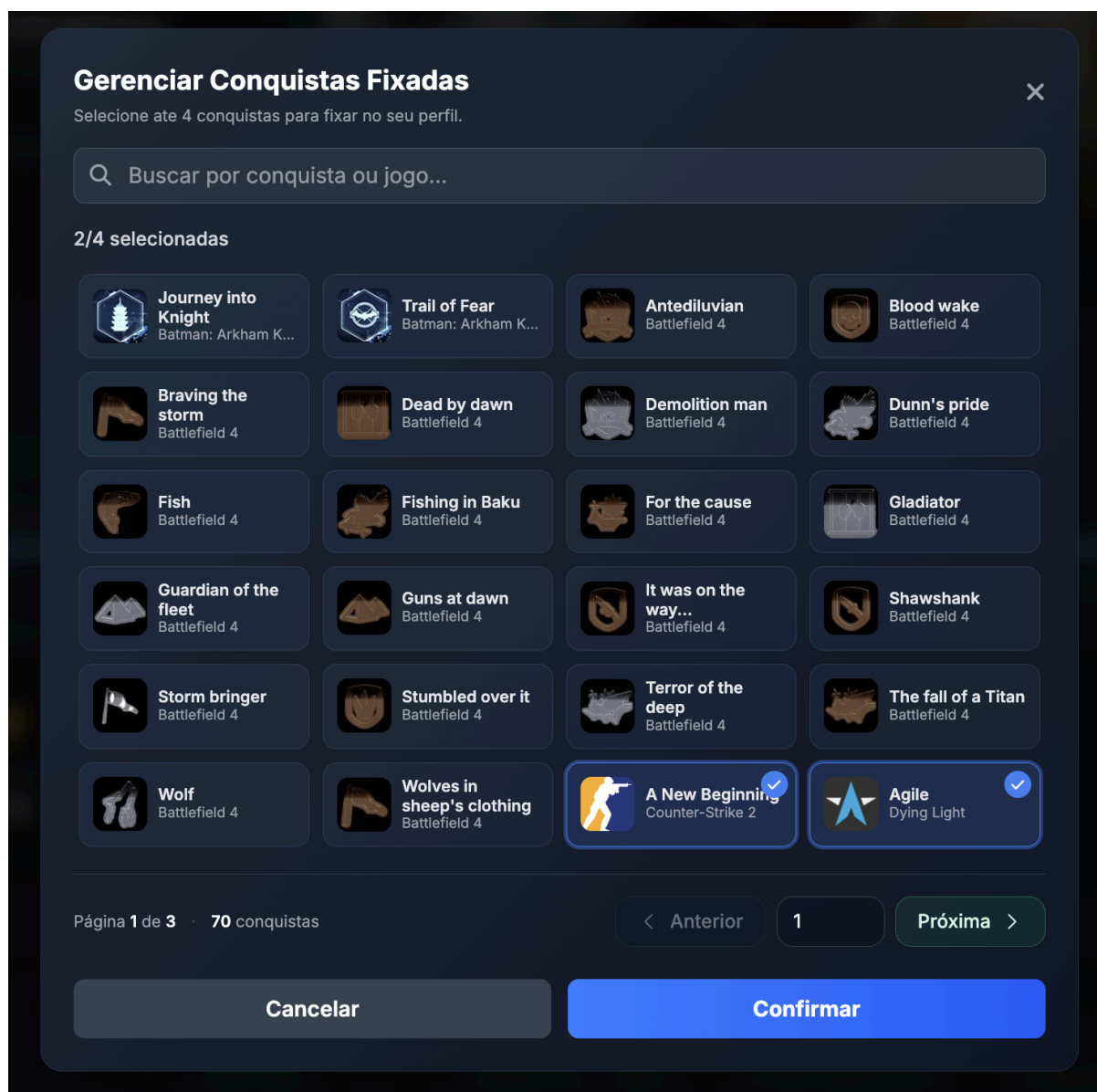
Conte um pouco sobre você e seus jogos favoritos

Máximo de 90 caracteres.

Cancelar Salvar Alteracoes

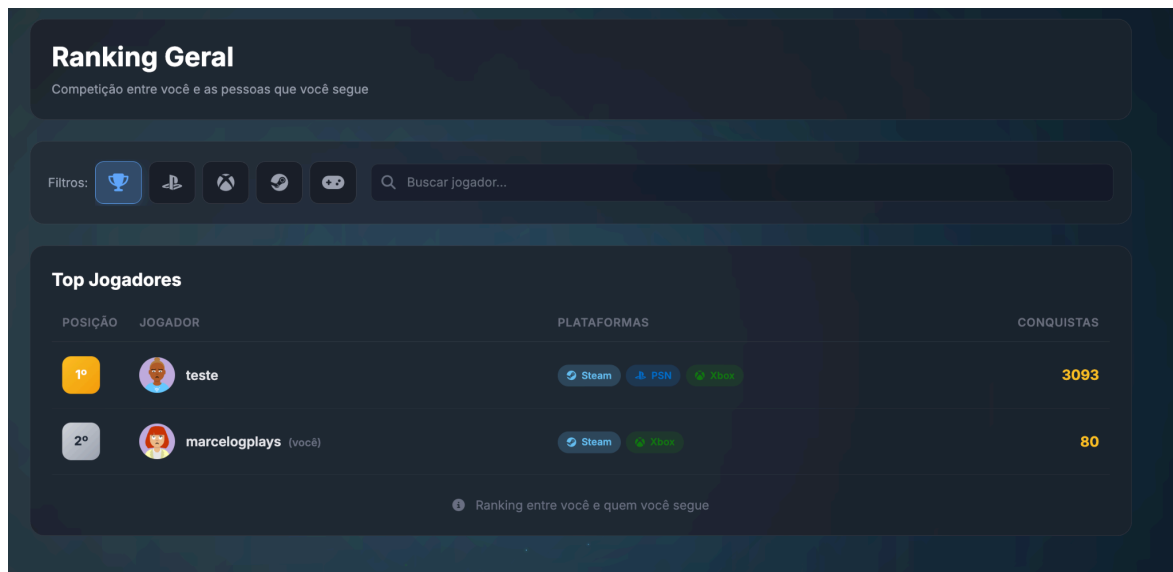
*Tela onde o usuário pode personalizar seu perfil*

Figura 7: Tela de personalizar troféus fixados



Presente na página de perfil, permite ao usuário visualizar todos os troféus que ele conquistou e fixar até 4 em seu perfil.

**Figura 8: Tela de ranking geral**



*Tela que exibe a classificação de jogadores no ranking geral*

**Figura 9: Tela de seguidores e seguindo**



*Tela que exibe os perfis que seguem o usuário e os perfis que ele segue, permitindo seguir ou deixar de seguir esses perfis.*

## 6. AMBIENTE E GUIA DE IMPLANTAÇÃO

### 6.1. REQUISITOS DO AMBIENTE

- **Sistema Operacional:** Linux Kernel 6.8 (ou superior) / Windows 10 (ou superior)
- **Git:** v2.53.0
- **Containers:** Docker v29.4.3, Docker Compose v5.1.3
- **Contas para extração de chaves e segredos:** [Google](#), [IGDB](#), [Steam](#), [Azure](#)

### 6.2. PROCESSO DE IMPLANTAÇÃO

#### Monolito Web:

1. Primeiro é necessário clonar o repositório para ter acesso em sua maquina:

```
git clone https://github.com/ajuteixeira/projeto_prisma.git
```

2. Após isso, navegue até a pasta do projeto:

```
cd projeto_prisma
```

3. Ao chegar a pasta raiz do projeto, vamos iniciar a configuração das variaveis de ambiente necessarias, primeiramente, copie o arquivo .env.example para um novo arquivo .env

```
cp .env.example .env
```

4. Após isso, acesse o arquivo com o editor de texto preferido:

```
nvim .env // nano .env // code .env
```

5. Configuração do gmail SMTP, serviço de envio de emails:

- 5.1. Crie uma conta no gmail

- 5.2. Acesse [myaccount.google.com/security](https://myaccount.google.com/security) e ative a Verificação em duas etapas

- 5.3. Acesse [myaccount.google.com/apppasswords](https://myaccount.google.com/apppasswords) e gere uma senha do app

- 5.4. No arquivo .env, preencha os campos:  
GMAIL\_USER=[suaconta@gmail.com](mailto:suaconta@gmail.com)  
GMAIL\_APP\_PASSWORD=suasenhadoapp
6. Após isso, vamos fazer a configuração do IGBD:
  - 6.1. Garanta que você possui uma conta [Twitch](#) com autenticação de dois fatores habilitada.
  - 6.2. Após isso, entre no site de [desenvolvedores twitch](#)
  - 6.3. E preencha o campo de nome com o nome desejado para o projeto, confirme que é um nome não existente ainda
  - 6.4. Adicione o redirect para OAuth com o ip abaixo:  
  
http://localhost:4000
  - 6.5. Selecione a categoria *website integration*
  - 6.6. Confirme que a opção confidential está selecionada e clique em criar
  - 6.7. Após ser redirecionado, selecione novamente a aplicação criada, e preencha o campo de IGDB\_CLIENT\_ID=
  - 6.8. Após isso, clique em gerar novo segredo e preencha o campo IGBD\_CLIENT\_SECRET=
7. Também é necessário realizar uma configuração relacionada a Azure, acesse [este link](#)
  - 7.1. Após garantir que o login foi realizado, acesse *Novo Registro*
  - 7.2. Adicione um nome para a aplicação, Selecione Api do tipo *Somente Contas pessoais*
  - 7.3. Em URI de redirecionamento, selecione web e adicione <http://localhost:4000/auth/xbox/callback>, clique em registrar.
  - 7.4. Selecione o app registrado e preencha XBOX\_CLIENT\_ID= com o campo ID do aplicativo (cliente)
  - 7.5. Dirija-se para certificados e segredos na barra lateral e clique em novo segredo
  - 7.6. Adicione uma descrição e selecione o tempo caso necessário, Preencha XBOX\_CLIENT\_SECRET= com as credenciais no campo VALOR
8. Por último, Garanta que possui uma conta steam e acesse [este link](#), Nomeie a chave, aceite os termos de uso e clique em registrar, caso necessário faça a confirmação via steam guard, copie o campo Key: e Preencha o campo STEAM\_API\_KEY=
9. Lembre de salvar as alterações no arquivo antes de finalizar a edição
10. Finalmente, é possível rodar o projeto com completa funcionalidade, inicialize o docker com o comando:

**Docker compose up --build -d ou Docker-compose up --build -d**

11. Por fim, é possível parar o projeto rodando:

Docker compose down -v Caso queira limpar o banco de dados ou

Docker compose down Caso queira persistir o banco de dados

### 6.3. ACESSO À APLICAÇÃO IMPLANTADA

- **URL da Aplicação:** <https://projeto-prisma.duckdns.org/>
- **Credenciais de Acesso (Perfil de Usuário teste):**
  - **Usuário:** *teste@teste.com*
  - **Senha:** *teste123123*

## 7. CONCLUSÃO

### 7.1. TRABALHOS FUTUROS

O ciclo de vida de uma aplicação web vai muito além da sua entrega inicial. Com o intuito de escalar a plataforma e entregar ainda mais valor aos usuários, mapeamos as seguintes evoluções para as próximas versões do Sistema Prisma:

- Desenvolvimento de uma versão mobile nativa para smartphones;
- Suporte para upload de mídias (capturas de tela);
- Inclusão de um botão de ação rápida para "Seguir" diretamente no card do perfil do usuário, otimizando a experiência de navegação e interação;
- Criação de um módulo de grupos e fóruns internos para unir jogadores com base em interesses ou jogos específicos em comum.

### 7.2. LIÇÕES APRENDIDAS

A construção do Sistema Prisma proporcionou um amadurecimento profundo para a equipe, evidenciando as particularidades de integrar múltiplas tecnologias em um único ambiente coeso. O principal desafio envolveu a comunicação com as APIs de terceiros (Steam, Xbox, PlayStation e RetroAchievements), devido a cada serviço possuir estruturas de dados, métodos de autenticação e limites de requisição completamente distintos. Na organização e condução do grupo, a transparência garantida pelo uso do Kanban via GitHub Projects evitou a sobreposição de tarefas durante os ciclos de desenvolvimento. Já a orquestração do ambiente com Docker foi vital para nivelar o acesso à infraestrutura, erradicando conflitos de sistema operacional e dependências locais.